

Digital Electronics

Complete Basic DLD (Digital Logic Design) Cheat Sheet

A detailed, exam-ready reference covering Number Systems, Logic Gates, Boolean Algebra, K-Maps, Combinational Circuits, Sequential Circuits, and Key Formulas — with worked examples, truth tables, and explanations for every concept.

#	Topic
1	Number Systems
2	Logic Gates
3	Boolean Algebra
4	Karnaugh Map (K-Map)
5	Combinational Circuits
6	Sequential Circuits
7	Key Formulas

1. Number Systems

A number system is a structured way of representing quantities using a fixed set of symbols (digits). Digital circuits are built from electronic switches that can only be in one of two states — ON or OFF — which is exactly why binary (base 2) is the natural language of computers.

1.1 The Four Common Number Systems

System	Base	Digits Used	Example
Binary	2	0, 1	1011
Octal	8	0 – 7	573
Decimal	10	0 – 9	925
Hexadecimal	16	0–9, A–F	3F9

Hexadecimal is popular in programming and memory addressing because 1 hex digit exactly represents 4 binary bits, making long binary strings much shorter and easier to read.

1.2 Conversions Between Systems

Binary to Decimal

Multiply each bit by 2 raised to its position (counted from the right, starting at 0), then add all the results.

Example: $1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11_{10}$

Decimal to Binary

Repeatedly divide the decimal number by 2, note down each remainder, and read the remainders from bottom to top.

Example: convert 13 to binary

$13 \div 2 = 6$ remainder 1

$6 \div 2 = 3$ remainder 0

$3 \div 2 = 1$ remainder 1

$1 \div 2 = 0$ remainder 1

Reading remainders bottom to top: **1101₂**

Binary to Octal

Group the bits into sets of 3 starting from the right (pad with leading zeros if needed), then convert each group to its octal digit.

Example: $101110_2 \rightarrow$ group as 101 110 $\rightarrow$ 5 6 $\rightarrow$ **56₈**

Binary to Hexadecimal

Group the bits into sets of 4 starting from the right, then convert each group to its hex digit.

Example: $10111010_2 \rightarrow$ group as 1011 1010 $\rightarrow$ B A $\rightarrow$ **BA₁₆**

Octal / Hexadecimal to Binary

Simply reverse the grouping process — expand each octal digit into 3 bits, or each hex digit into 4 bits.

1.3 Binary Arithmetic

Binary Addition Rules: 0+0=0, 0+1=1, 1+0=1, 1+1=10 (write 0, carry 1).

Binary Subtraction Rules: 0-0=0, 1-0=1, 1-1=0, 0-1=1 (borrow 1 from next column).

Example addition: 1011 + 1101

```
1011
+ 1101
-----
```

11000 (carry propagates through each column)

1.4 Signed Number Representations

Digital systems need a way to represent negative numbers using only 0s and 1s. There are three common methods:

Sign-Magnitude: The leftmost (most significant) bit indicates the sign — 0 for positive, 1 for negative — while the remaining bits hold the magnitude (absolute value). Simple to understand but makes arithmetic circuits more complex.

1's Complement: To represent a negative number, flip every bit of its positive binary form.

Example: +5 = 0101, so -5 in 1's complement = 1010.

Drawback: it produces two representations of zero (0000 and 1111), which complicates arithmetic hardware and wastes a code.

2's Complement: Take the 1's complement of the number, then add 1. This is the standard used in virtually all real computers.

Example: +5 = 0101 → 1's complement = 1010 → add 1 = 1011 (this is -5).

Advantage: only one representation of zero, and subtraction can be performed using the same adder hardware as addition (by adding the 2's complement).

Range for n-bit 2's complement: $-2^{(n-1)}$ to $+2^{(n-1)} - 1$. For example, an 8-bit signed number ranges from -128 to +127.

1.5 BCD (Binary Coded Decimal)

Instead of converting an entire decimal number into pure binary, each individual decimal digit (0-9) is converted separately into its own 4-bit binary code.

Example: decimal 57 in BCD → 5 = 0101, 7 = 0111 → **0101 0111**

Note this is different from pure binary 57, which is 111001.

Invalid BCD codes: 1010 through 1111 (these don't represent any decimal digit).

Used in digital clocks, calculators, and any display where individual decimal digits need to be shown separately.

1.6 Gray Code

A binary numbering system where only a single bit changes between any two successive values. This property reduces errors in mechanical/rotary position sensors and is also the basis for how K-map cells are ordered.

Rule to convert Binary to Gray: keep the MSB the same, then each subsequent Gray bit = XOR of the current binary bit and the previous binary bit.

Example: Binary 1011 → Gray = 1, $(1 \oplus 0)=1$, $(0 \oplus 1)=1$, $(1 \oplus 1)=0$ → **1110**

1.7 Excess-3 Code

Obtained by adding 3 (0011) to the 4-bit BCD value of each decimal digit. It is a self-complementing code, meaning the 9's complement of a decimal digit can be obtained just by inverting all the bits — useful in some arithmetic circuits.

Example: decimal 4 → BCD 0100 → add 0011 → Excess-3 = **0111**

1.8 More Worked Conversion Examples

Decimal	Binary	Octal	Hexadecimal
10	1010	12	A
25	11001	31	19
64	1000000	100	40
255	11111111	377	FF

Tip: memorizing this table for small numbers (0-16) speeds up hex/binary conversions significantly during exams, since every hex digit maps to exactly 4 binary bits.

1.9 Binary Subtraction Using 2's Complement

Rather than borrowing bit-by-bit, digital circuits perform $A - B$ by computing $A + (2\text{'s complement of } B)$, then discarding any final overflow carry.

Example: $7 - 3$ using 4-bit numbers $\rightarrow 7 = 0111, 3 = 0011$

2's complement of 3: 1's complement (1100) + 1 = 1101

Add: $0111 + 1101 = 1\ 0100 \rightarrow$ discard the overflow carry (leftmost 1)

Result: $0100 = \text{decimal } 4$ (which correctly equals $7 - 3$)

1.10 Binary Multiplication and Division (brief)

Multiplication follows the same shift-and-add process as long multiplication in decimal, just using binary digits.

Division follows the same repeated-subtraction / shift process as long division in decimal. Digital circuits typically implement these using adder and shift-register hardware rather than dedicated multiply/divide gates.

2. Logic Gates

Logic gates are the fundamental building blocks of every digital circuit — tiny electronic switches that accept one or more binary inputs and produce a single binary output according to a fixed logical rule.

2.1 AND Gate

Output is 1 only if **all** inputs are 1. Behaves like a series circuit: both switches must be closed for current to pass through.

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

2.2 OR Gate

Output is 1 if **any** input is 1. Behaves like a parallel circuit: current flows if either path is closed.

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

2.3 NOT Gate (Inverter)

A single-input gate that simply flips the input: 0 becomes 1, and 1 becomes 0.

A	Output
0	1
1	0

2.4 NAND Gate

AND followed by NOT. Output is 1 in every case except when both inputs are 1.

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0

2.5 NOR Gate

OR followed by NOT. Output is 1 only when both inputs are 0.

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

2.6 XOR (Exclusive OR) Gate

Output is 1 only when the inputs are **different**. Extremely useful for comparing bits and forms the core of adder circuits.

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

2.7 XNOR Gate

The opposite of XOR — output is 1 when the inputs are the **same**.

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

2.8 Universal Gates — NAND and NOR

NAND and NOR are called "universal gates" because any logic function — AND, OR, NOT, or any combination of them — can be built using only NAND gates (or only NOR gates).

NOT using NAND: tie both inputs of a NAND gate together.

AND using NAND: a NAND gate followed by a NOT (built from another NAND).

OR using NAND: invert both inputs first, then feed them into a NAND gate (this follows directly from De Morgan's theorem).

This matters practically because manufacturers can mass-produce a single gate type (NAND) and use it to build entire complex circuits, which reduces cost and simplifies chip design.

2.9 A Note on IC Logic Families

TTL (Transistor-Transistor Logic): Fast switching speed, but relatively higher power consumption.

CMOS (Complementary Metal-Oxide-Semiconductor): Very low power consumption and high noise immunity, which is why it's used in almost all modern chips.

Fan-out: the maximum number of gate inputs a single gate's output can safely drive.

Propagation delay: the time taken for a change at the input to be reflected at the output.

Family	Speed	Power Use	Noise Immunity
TTL	Fast	Higher	Moderate
CMOS	Moderate-Fast	Very Low	High
ECL	Fastest	Highest	Lower

2.10 Building Basic Gates from NAND Only

Because NAND is universal, here is exactly how each basic gate is derived from it:

Target Gate	NAND-Only Construction
NOT (A')	Tie both inputs of a single NAND gate to A
AND ($A.B$)	NAND(A,B) followed by a NOT (built from another NAND)
OR ($A+B$)	NOT A and NOT B first, then feed both into a NAND gate
NOR ($(A+B)'$)	Take the OR construction above, then invert once more

The same logic applies symmetrically for building every gate from NOR only, just with OR/AND roles swapped.

3. Boolean Algebra

A mathematical system, developed by George Boole, used to analyze and simplify logic expressions. Simplifying an expression before building the circuit means fewer gates are needed — resulting in cheaper, faster, and more reliable hardware.

3.1 Basic Postulates

Closure: the result of any Boolean operation on Boolean values is always a Boolean value (0 or 1).

Identity: $A+0=A$ and $A \cdot 1=A$.

Complement: $A+A'=1$ and $A \cdot A'=0$.

3.2 Fundamental Laws (with meaning)

Commutative Law: $A+B = B+A$, $A \cdot B = B \cdot A$

The order in which you write the variables doesn't matter.

Associative Law: $(A+B)+C = A+(B+C)$

The way variables are grouped with brackets doesn't matter.

Distributive Law: $A(B+C) = AB+AC$

Works exactly like distributing multiplication over addition in ordinary algebra.

Idempotent Law: $A+A=A$, $A \cdot A=A$

Repeating the same variable doesn't add any new information.

Absorption Law: $A+AB=A$, $A(A+B)=A$

If A is already true, ANDing or ORing it with extra terms involving A doesn't change the result.

Involution Law: $(A')' = A$

Complementing a variable twice returns the original variable.

Consensus Theorem: $AB + A'C + BC = AB + A'C$

The BC term is redundant and can always be eliminated — useful for extra simplification.

3.3 De Morgan's Theorems

Theorem 1: $(A+B)' = A' \cdot B'$ — the complement of an OR expression equals the AND of the individual complements.

Theorem 2: $(A \cdot B)' = A' + B'$ — the complement of an AND expression equals the OR of the individual complements.

These theorems are what allow you to convert freely between AND/OR/NOT-based circuits and NAND/NOR-only circuits — extremely important for circuit implementation using universal gates.

3.4 Worked Simplification Example

Simplify: $Y = AB + AB' + A'B$

Step 1: Group first two terms $\rightarrow AB + AB' = A(B+B') = A \cdot 1 = A$

Step 2: Expression becomes $\rightarrow A + A'B$

Step 3: Apply absorption-style expansion $\rightarrow A + A'B = A + B$

Final simplified result: **$Y = A + B$**

3.5 Duality Principle

Every Boolean expression has a "dual" — obtained by swapping every AND with OR, every OR with AND, and every 0 with 1 (variables stay unchanged). If an expression is a valid Boolean identity, its dual is automatically valid too.

Example: the dual of $A + (B \cdot C) = (A+B) \cdot (A+C)$ is obtained by swapping $+$ and $\cdot$ throughout — giving back the distributive law in its other form, which confirms both are valid identities.

3.6 Verifying De Morgan's Theorem with a Truth Table

A quick way to confirm $(A+B)' = A' \cdot B'$ is to build the truth table for both sides and check every row matches:

A	B	$(A+B)'$	$A' \cdot B'$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

Since both columns match in every row, the identity is confirmed. This truth-table check is a reliable way to verify any Boolean law when in doubt.

3.7 Another Worked Simplification

Simplify: $Y = A'BC + AB'C + ABC' + ABC$

Group terms sharing common factors:

$ABC' + ABC = AB(C' + C) = AB$

$A'BC + AB'C = C(A'B + AB')$ — doesn't reduce further alone, keep as is

Combine: $Y = AB + C(A'B + AB') = AB + BC + AC$ (after full expansion and reapplying consensus)

Final simplified result: $Y = AB + BC + AC$

4. Karnaugh Map (K-Map)

A visual, grid-based method for simplifying Boolean expressions — much faster than repeatedly applying algebra laws once you're comfortable reading the grid.

4.1 How a K-Map Works

Each cell in the grid corresponds to exactly one row of the truth table — one specific combination of input values. The cells are deliberately arranged so that any two adjacent cells (including wrap-around edges) differ by only one bit. This arrangement is called Gray code ordering, and it's precisely what makes the grouping technique mathematically valid — adjacent 1s can always be combined into a simpler term.

You fill the map by placing a "1" in every cell where the function's output is true (and "0" or leave blank where it's false), based on the truth table.

4.2 Map Sizes

Variables	Grid Layout	Total Cells
2	2 x 2	4
3	2 x 4	8
4	4 x 4	16
5	4 x 8 (or two 4x4 maps)	32

4.3 Grouping Rules

1. Circle groups of adjacent 1s in sizes that are powers of 2 only: 1, 2, 4, 8, or 16 cells.
2. Larger groups produce simpler terms, because more variables cancel out of the final expression — always try to make the biggest legal group possible.
3. Groups may wrap around the edges of the map (top row is adjacent to bottom row, leftmost column is adjacent to rightmost column).
4. The same cell can be included in more than one group if it helps produce simpler overall terms.
5. Every 1 in the map must be covered by at least one group.

4.4 Don't-Care Conditions

Sometimes certain input combinations will never actually occur in practice (or their output doesn't matter). These cells are marked with an "X" and can be treated as either 0 or 1 — whichever helps form a larger, simpler group. This often leads to a more minimal final expression.

4.5 SOP vs POS

SOP (Sum of Products): Group the 1s in the map, and write the result as OR-ed AND-terms (a "sum" of "product" terms). This is the most common, beginner-friendly approach and directly matches how AND-OR circuits are built.

POS (Product of Sums): Group the 0s instead, and write the result as AND-ed OR-terms. Both methods produce an equally correct, minimized circuit — POS is sometimes preferred when the map has more 0s than 1s.

4.6 Worked Example — 3-Variable K-Map

Simplify $F(A,B,C) = \Sigma(1,3,5,7)$ — meaning the function is 1 for minterms 1, 3, 5, and 7 (in binary: 001, 011, 101, 111) and 0 elsewhere.

Notice that in every one of these minterms, $C = 1$, while A and B both take all possible combinations. Plotting these on a 3-variable K-map produces one large group of all four cells where $C=1$.

Grouping these 4 adjacent cells eliminates both A and B entirely, leaving only the variable that stayed constant across the group.

Simplified result: $F = C$

4.7 Worked Example — 4-Variable K-Map

Simplify $F(A,B,C,D) = \Sigma(0,2,8,10)$ — all four minterms have $B=0$ and $D=0$, while A and C vary freely.

These four cells form a valid group of 4 (using wrap-around adjacency between the map's corners), so A and C cancel out, leaving only the two variables that stayed constant.

Simplified result: $F = B'D'$

5. Combinational Circuits

Circuits whose output depends only on the present combination of inputs — there is no memory of past inputs involved. They are built directly by wiring together logic gates.

5.1 Half Adder

Adds two single bits, A and B, producing a Sum and a Carry.

Sum = $A \oplus B$ (XOR gate) Carry = $A \cdot B$ (AND gate)

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Limitation: a half adder cannot accept a carry coming in from a previous addition stage, so on its own it's only useful for adding the least significant bit of a multi-bit number.

5.2 Full Adder

Adds two bits (A, B) plus a carry-in (C_{in}) from the previous stage, which makes it usable in a chain for adding multi-bit binary numbers.

Sum = $A \oplus B \oplus C_{in}$

Carry-out = $AB + C_{in}(A \oplus B)$

A full adder can be built using two half adders and an OR gate: the first half adder adds A and B, the second half adder adds that result to C_{in} , and the OR gate combines the two carry outputs.

5.3 Half Subtractor

Subtracts one bit (B) from another (A), producing a Difference and a Borrow.

Difference = $A \oplus B$ Borrow = $A' \cdot B$

5.4 Full Subtractor

Same concept as the full adder, but for subtraction — it additionally accepts a borrow-in from the previous stage, allowing multi-bit subtraction.

5.5 Multiplexer (MUX)

A data selector circuit. It has several data input lines, a set of select lines, and a single output. The binary value on the select lines determines exactly which one data input gets connected through to the output.

A MUX with n select lines can choose from 2^n data inputs (for example, 2 select lines → a 4-to-1 MUX).

Applications: data routing, parallel-to-serial conversion, and implementing Boolean functions directly (any function of n variables can be built using a single 2^n -to-1 MUX).

Truth table for a 4-to-1 MUX (2 select lines, S1 S0):

S1	S0	Output =
0	0	I0

0	1	I1
1	0	I2
1	1	I3

5.6 Demultiplexer (DEMUX)

The reverse of a MUX: it has a single input line, a set of select lines, and multiple output lines. The select lines determine which single output receives the incoming input signal, while all other outputs stay inactive.

5.7 Encoder

Takes 2^n input lines (where, normally, only one input is active at any given time) and converts which input is active into an n-bit binary output code. A common example is an octal-to-binary encoder (8 inputs, 3-bit output).

Priority Encoder: A more practical version that resolves the situation where multiple inputs are active simultaneously — it outputs the code corresponding to whichever active input has the highest assigned priority.

5.8 Decoder

The reverse of an encoder: takes n binary input lines and activates exactly one of 2^n output lines, based on the specific binary combination presented at the input. Used heavily in memory address decoding and driving displays (e.g. BCD-to-7-segment decoders).

5.9 Comparator

Compares two binary numbers, A and B, and produces three possible outputs: $A > B$, $A = B$, or $A < B$. The equality check is commonly built using XNOR gates, since XNOR outputs 1 only when corresponding bits match.

5.10 Parity Generator / Checker (bonus)

A simple error-detection mechanism. An extra "parity bit" is added to a data word so that the total count of 1-bits (including the parity bit) is always even (even parity) or always odd (odd parity). At the receiving end, a parity checker recomputes this and flags a mismatch as a transmission error.

5.11 Worked Example — 2-to-4 Decoder Truth Table

A 2-to-4 decoder has 2 input lines (A, B) and 4 output lines (Y0-Y3), with exactly one output active (1) for each input combination:

A	B	Y0	Y1	Y2	Y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

5.12 Code Converters

Circuits designed specifically to convert data from one code format to another. Common examples:

Binary-to-Gray converter: built from a chain of XOR gates, following the rule described in Section 1.6.

BCD-to-7-Segment decoder: converts a 4-bit BCD input into the correct combination of segment signals needed to

visually display that digit on a 7-segment LED display.

BCD-to-Excess-3: adds 0011 to the BCD input to produce the self-complementing Excess-3 code.

6. Sequential Circuits

Unlike combinational circuits, sequential circuits have memory — their output depends on both the current inputs and the circuit's past state (stored internally). This memory property is what enables counting, data storage, and timing-controlled behavior.

6.1 Latches vs Flip-Flops

Latch: a level-triggered memory element — it responds to its inputs continuously, for as long as an enable signal stays active.

Flip-Flop: an edge-triggered memory element — it only updates its output at the precise moment the clock signal transitions (rising or falling edge), which makes circuit timing far more predictable and controllable.

6.2 SR Flip-Flop (Set-Reset)

The most basic flip-flop, built from two cross-coupled NAND or NOR gates.

S=1, R=0 → output is SET to 1.

S=0, R=1 → output is RESET to 0.

S=0, R=0 → output HOLDS its previous value.

S=1, R=1 → **invalid / undefined state** — both outputs try to go to the same value, which breaks the flip-flop's normal complementary behavior. This is considered a forbidden input combination.

6.3 D Flip-Flop (Data / Delay)

The simplest flip-flop to use in practice — whatever logic value is present on the single D input gets transferred straight to the output at the next active clock edge. Because there's only one input, the invalid-state problem of the SR flip-flop is completely eliminated. Extremely common in registers and memory elements.

6.4 JK Flip-Flop

An improvement over the SR flip-flop that explicitly defines a useful behavior for the previously-invalid input combination.

J=0, K=0 → HOLD (no change)

J=0, K=1 → RESET (output becomes 0)

J=1, K=0 → SET (output becomes 1)

J=1, K=1 → **TOGGLE** (output flips to the opposite of its current value) — this is what fixes SR's invalid state, and is why JK is often called the "universal" flip-flop.

6.5 T Flip-Flop (Toggle)

A simplified version of the JK flip-flop with a single input, T (obtained by tying J and K together).

T=0 → output HOLDS its value.

T=1 → output TOGGLES on every clock pulse.

Because of this toggling behavior, T flip-flops are the natural building block for binary counter circuits.

6.6 Characteristic Tables (Summary)

A characteristic table shows the next output state (Q-next) for every possible combination of inputs and the current state (Q).

Flip-Flop	Inputs = 0,0	Inputs = 0,1	Inputs = 1,0	Inputs = 1,1
SR	Hold	Reset (0)	Set (1)	Invalid

JK	Hold	Reset (0)	Set (1)	Toggle
D	Q-next = D (single input)			
T	Hold (T=0)	Toggle (T=1)		

Excitation tables work the opposite way — given a desired state transition (current Q to next Q), they tell you what input values are needed to achieve it. These are used when designing a sequential circuit to match a required state diagram.

6.7 Mealy vs Moore Machines (Finite State Machines)

Sequential circuits are often modeled formally as finite state machines (FSMs):

Moore Machine: the output depends only on the current state, not directly on the input. Outputs change only at clock edges, making the design generally simpler and glitch-free.

Mealy Machine: the output depends on both the current state AND the current input. This often results in fewer states for the same task, but the output can change asynchronously with input changes.

6.8 Registers

A register is simply a group of flip-flops wired together to store multiple bits as a single unit (e.g. an 8-bit register stores one byte of data, using eight D flip-flops in parallel).

Shift Registers are a special type of register that moves the stored bits one position left or right on every clock pulse. They're classified by how data enters and exits:

Type	Data In	Data Out	Typical Use
SISO	Serial	Serial	Time delay lines
SIPO	Serial	Parallel	Serial-to-parallel conversion
PISO	Parallel	Serial	Parallel-to-serial conversion
PIPO	Parallel	Parallel	Temporary data storage / buffering

6.9 Counters

Sequential circuits built from flip-flops that cycle through a defined sequence of binary states — typically counting up or down — on every clock pulse.

Asynchronous (Ripple) Counter: Only the first flip-flop is connected to the actual clock signal; each subsequent flip-flop is triggered by the output of the previous one. Simple and cheap to build, but slower, because the triggering effect "ripples" through the chain one flip-flop at a time, creating cumulative propagation delay.

Synchronous Counter: Every flip-flop shares the exact same clock signal, so all of them change state at precisely the same instant. This removes the ripple delay, making synchronous counters significantly faster and more suitable for high-speed applications, though they require slightly more complex control logic.

Modulo-N Counter: A counter designed to count from 0 up to N-1 and then reset back to 0, repeating the cycle (e.g. a mod-10 counter cycles through 0-9, used in digital clocks and decade counters).

Up/Down Counter: Can be configured, using a control input, to count either upward or downward through its sequence.

6.10 Ring and Johnson Counters

Ring Counter: A shift register where the output is fed back into the input, circulating a single 1 (or 0) around the loop of flip-flops. An N flip-flop ring counter has N unique states.

Johnson Counter (Twisted Ring): Same idea, but the complemented output is fed back instead. This doubles the number of unique states to $2N$ for N flip-flops, making it more efficient than a plain ring counter.

6.11 General Steps for Designing a Sequential Circuit

1. Draw the state diagram showing every state and the transitions between them based on inputs.
2. Convert the state diagram into a state table (current state, input, next state, output).
3. Assign binary codes to each state (state assignment).
4. Use the excitation table of the chosen flip-flop type to determine the required flip-flop inputs for each transition.
5. Derive simplified Boolean expressions for the flip-flop inputs (using K-maps).
6. Draw the final logic circuit using the flip-flops and the derived gate logic.

Solved Practice Examples

A short set of worked problems pulling together concepts from across the cheat sheet — useful for quick self-testing before an exam.

Example 1 — Number Systems

Q: Find the 2's complement of the 6-bit number 010110.

Step 1 — 1's complement (flip every bit): 101001

Step 2 — add 1: $101001 + 1 = \mathbf{101010}$

Example 2 — Boolean Algebra

Q: Simplify $Y = (A+B)(A+B')$.

Step 1 — Expand: $AA + AB' + AB + BB'$

Step 2 — $AA = A$, $BB' = 0$, so: $A + AB' + AB$

Step 3 — Factor: $A(1 + B' + B) = A \cdot 1$

Result: $Y = A$

Example 3 — Combinational Circuit

Q: What is the Sum and Carry output of a Full Adder for $A=1$, $B=1$, $C_{in}=1$?

Sum = $A \oplus B \oplus C_{in} = 1 \oplus 1 \oplus 1 = 1$

Carry = $AB + C_{in}(A \oplus B) = (1 \cdot 1) + 1 \cdot (1 \oplus 1) = 1 + 0 = 1$

Result: Sum = 1, Carry = 1 (binary total = 11, i.e. decimal 3)

Example 4 — Sequential Circuit

Q: A JK flip-flop currently holds $Q=0$. If $J=1$ and $K=1$ are applied on the next clock edge, what is the new value of Q ?
 $J=1$, $K=1$ corresponds to the Toggle condition, so Q flips from its current value.

Result: Q becomes 1

Example 5 — Multiplexer Sizing

Q: How many select lines are needed to build a MUX that can choose between 16 data inputs?

$16 = 2^4$, so $n = 4$ select lines are required.

Result: 4 select lines → 16-to-1 MUX

7. Key Formulas to Remember

Formula	What It Gives You
$2^n - 1$	Maximum unsigned value storable in n bits
2^n	Number of inputs a MUX can handle with n select lines
2^n	Number of outputs a decoder produces with n input lines
n	Select lines needed for a 2^n -to-1 MUX or 1-to- 2^n DEMUX
$-2^{(n-1)}$ to $+2^{(n-1)} - 1$	Range of an n-bit signed 2's complement number
2^n	Total number of distinct states / addresses for n bits

Quick Recap

- Number Systems — how quantities are represented and converted between binary, octal, decimal, and hex.
- Logic Gates — the basic switches (AND, OR, NOT, NAND, NOR, XOR, XNOR) that all digital circuits are built from.
- Boolean Algebra — the rules used to simplify logic expressions before building a circuit.
- K-Map — a visual shortcut for the same simplification process.
- Combinational Circuits — gate-based circuits with no memory (adders, subtractors, MUX, DEMUX, encoders, decoders, comparators).
- Sequential Circuits — circuits with memory (flip-flops, registers, counters) that depend on past state as well as current input.

Common Mistakes to Avoid

- Confusing 1's complement with 2's complement — remember 2's complement always requires the extra "+1" step after flipping the bits.
- Forgetting that K-map cells wrap around at the edges — corner and border cells can still be adjacent to each other.
- Mixing up SR flip-flop's invalid state with JK's toggle state — they look similar (both inputs = 1) but behave completely differently.
- Forgetting the OR gate in a full adder's carry-out logic when building it from two half adders.
- Writing NAND/NOR truth tables backwards — always work out the base gate (AND/OR) first, then invert every output.
- Assuming BCD is the same as plain binary — they diverge for any number with digits above 9 in one place.
- Forgetting that De Morgan's theorem swaps the operator ($\text{AND} \leftrightarrow \text{OR}$) in addition to distributing the complement.

End of cheat sheet — complete basic DLD (Digital Logic Design) subject coverage.